Software Requirement Specification for A Smart Hospital Appointment and Record Management System

Authors

Atharva Patil
Department of Computer Science and Engineering (Artificial Intelligence)
Vishwakarma Institute of Technology, Pune, India
Email: atharva.patil23@vit.edu

Swaraj Patil
Department of Computer Science and Engineering (Artificial Intelligence)
Vishwakarma Institute of Technology, Pune, India
Email: swaraj.patil23@vit.edu

Kavish Paraswar
Department of Computer Science and Engineering (Artificial Intelligence)
Vishwakarma Institute of Technology, Pune, India
Email: kavish.paraswar23@vit.edu

Shwetkumar Ramrteke
Department of Computer Science and Engineering (Artificial Intelligence)
Vishwakarma Institute of Technology, Pune, India
Email: shwetkumar.ramrteke232@vit.edu

Abstract

Healthcare appointment and record management systems have become an important part of modern hospital operations. Manual scheduling, paper-based medical records, and disconnected communication between patients, doctors, and administrators can cause delays, data inconsistency, and poor visibility into patient care activities. A smart hospital platform reduces these problems by providing a centralized digital system for appointment booking, doctor availability, medical record management, prescriptions, notifications, and administrative reporting.

This paper presents a detailed Software Requirement Specification (SRS) for a Smart Hospital Appointment and Record Management System. The proposed system is a role-based full-stack web application that supports patient self-service booking, doctor scheduling, real-time appointment lifecycle management, clinical record creation, prescription storage, notification delivery, and admin-level monitoring. The system uses secure authentication and authorization so that patients, doctors, and administrators access only the features and data permitted for their roles.

The specification defines the system purpose, scope, stakeholders, functional requirements, non-functional requirements, constraints, and hardware and software requirements. It also explains the requirement engineering approach used to identify, analyze, classify, and document the requirements. The document is intended to provide a clear foundation for design, development, testing, validation, and future enhancement of the hospital management platform.

Keywords: Software Requirement Specification, Smart Hospital System, Appointment Management, Medical Records, Prescriptions, Requirement Engineering, Healthcare Software, Role-Based Access Control

1 Introduction

Hospitals and clinics manage a large number of daily interactions between patients, doctors, and administrative staff. These interactions include patient registration, doctor discovery, appointment booking, visit management, clinical documentation, prescriptions, notifications, and operational reporting. When these activities are handled manually or through disconnected tools, the hospital may face appointment conflicts, delayed communication, misplaced records, and limited visibility into the current workload.

The Smart Hospital Appointment and Record Management System is designed to provide a unified digital platform for these essential healthcare workflows. The system enables patients to register, log in, view real doctors, select available doctor slots, book appointments, reschedule or cancel bookings, and view their medical records and prescriptions. Doctors can manage availability, view upcoming visits, complete appointments, create medical records, and issue prescriptions. Administrators can monitor doctors, patients, appointments, and summary reports.

From a software engineering perspective, the system represents a practical healthcare application that combines authentication, role-based access control, relational data modeling, REST APIs, appointment lifecycle management, and secure record access. A well-defined SRS is necessary because healthcare systems must handle sensitive information, accurate scheduling, and controlled access to records.

This document describes the requirements for the proposed system in a structured manner. It focuses on what the system should do, how different users should interact with it, and what quality constraints must be satisfied to make the software reliable, usable, maintainable, and secure.

2 Background and Related Context

Many healthcare organizations still depend on front-desk scheduling, paper registers, separate prescription files, and manual communication. Such processes can work for small workloads, but they become difficult to manage when patient volume increases. Doctors need a clear view of their upcoming visits, patients need reliable information about available slots, and administrators need accurate reports for decision-making.

Modern hospital systems use web-based and database-backed platforms to improve these workflows. A digital system can store structured information about users, doctors, patients, appointments, availability slots, medical records, prescriptions, notifications, and reports. It can also enforce rules such as preventing unauthorized record access, avoiding double booking of doctor slots, and allowing only doctors to create clinical records.

The proposed project follows this direction by using a React frontend, an Express backend API, Sequelize models, MySQL persistence, JWT authentication, bcrypt password hashing, and role-based middleware. The system is developed incrementally so that authentication, appointments, records, prescriptions, notifications, and reports can be delivered and tested as separate but connected modules.

3 Problem Statement

Designing a hospital appointment and record management system is challenging because the software must coordinate multiple users, sensitive data, and time-based operations. Patients require a simple booking experience, doctors require accurate visit queues and record tools, and administrators require reliable monitoring. Without a structured software system, hospitals may face the following problems:

- Appointment conflicts caused by manual slot allocation or duplicate bookings.
- Difficulty in finding real doctor availability and selecting valid appointment times.
- Delayed access to patient history, medical records, and prescriptions.
- Weak control over who can view or modify sensitive healthcare data.
- Limited administrative visibility into appointment status, doctors, patients, and reports.
- Poor traceability between requirements, implementation, and testing.

The problem addressed in this SRS is the need for a clear, comprehensive, and human-readable specification for a real database-backed hospital web application. The specification must define the system boundaries, user roles, functional behavior, quality attributes, constraints, and platform requirements so that development and testing can be performed with clarity.

4 System Overview

4.1 Purpose of the System

The primary purpose of the Smart Hospital Appointment and Record Management System is to provide a secure, reliable, and user-friendly platform that connects patients, doctors, and administrators. The system aims to reduce manual scheduling effort, prevent booking conflicts, improve medical record availability, and support hospital-level monitoring through digital workflows.

The system also supports doctor availability management. Instead of allowing patients to select any random appointment time, the platform provides booking based on real availability slots created by doctors. This makes the appointment workflow more accurate and closer to a real hospital scenario.

4.2 Scope of the System

The scope of the system includes:

- Patient, doctor, and administrator registration and login.
- Secure JWT-based authentication and role-based protected access.
- Doctor listing with specialization, qualification, experience, and consultation fee.
- Doctor availability slot creation and deletion.
- Patient appointment booking using real available slots.
- Appointment listing, rescheduling, cancellation, deletion, and completion.
- Doctor visit workflow with medical record creation and optional prescription creation.
- Patient access to medical records and prescriptions.
- Notification support for appointment and record-related events.
- Administrative summary reports for appointments, doctors, and specializations.

4.3 Stakeholders and User Classes

The system serves the following stakeholders:

- **Patients:** Users who book appointments, manage bookings, and view medical records.
- **Doctors:** Users who manage availability, consult patients, create records, and issue prescriptions.
- **Administrators:** Users who monitor hospital activity, reports, doctors, and patients.
- **Hospital Staff:** Operational users who benefit from reduced manual scheduling work.

- **Academic Evaluators:** Reviewers who validate the project against software engineering principles.

5 Requirement Engineering Methodology

Requirement Engineering (RE) is a disciplined process used to identify, analyze, document, validate, and manage the requirements of a software system. In a healthcare application, requirement engineering is especially important because the system deals with sensitive patient information, role-based permissions, appointment timing, and clinical documentation. Any ambiguity in requirements can lead to incorrect access, missed appointments, or incomplete patient records.

In this project, the requirement engineering methodology follows a structured approach aligned with standard software engineering practices. Requirements are derived from hospital workflows, stakeholder goals, and the implemented project modules. The process emphasizes clarity, feasibility, testability, and traceability.

5.1 System Identification and Domain Understanding

The first step was identifying the system as a smart hospital appointment and record management platform. The domain includes patient registration, doctor scheduling, appointment booking, visit completion, clinical record storage, prescription management, notifications, and administrative reporting. Understanding this domain helped define core entities such as users, patients, doctors, specializations, appointments, availability slots, medical records, prescriptions, and notifications.

Domain understanding also helped identify important constraints. For example, a patient should not book an appointment in the past, a doctor should not be double-booked for the same slot, and a patient should not access another patient's medical record. These constraints directly influence the functional and non-functional requirements.

5.2 Stakeholder Identification and Analysis

Stakeholders were identified based on direct and indirect interaction with the system. Patients are primary users because they initiate bookings and consume healthcare services. Doctors are primary users because they manage availability, appointments, records, and prescriptions. Administrators monitor the system and require summary information for hospital operations. Hospital staff benefit from reduced manual workload and more consistent data.

Each stakeholder has a different goal. Patients expect simple booking and easy access to their records. Doctors expect accurate visit queues and efficient record entry. Administrators expect reliable reporting and system visibility. These goals are reflected in the role-specific requirements.

5.3 Requirement Elicitation

Requirements were elicited by analyzing common hospital workflows and reviewing the features expected in an appointment and record management system. The elicitation process considered the normal flow of patient registration, doctor search, slot selection, appointment booking, consultation, medical record creation, prescription issue, and patient record viewing.

Exceptional scenarios were also considered. These include appointment cancellation, rescheduling, deleting a booking, unavailable doctor slots, unauthorized record access, missing patient or doctor profiles, and slot conflict handling. Considering such scenarios improves requirement completeness and helps guide backend validation.

5.4 Requirement Analysis and Classification

After elicitation, requirements were analyzed to remove duplication, resolve ambiguity, and classify behavior. Requirements were separated into functional requirements and non-functional requirements. Functional requirements describe system behavior such as login, booking, rescheduling, record creation, prescription creation, and report viewing. Non-functional requirements describe quality expectations such as security, reliability, usability, maintainability, performance, and privacy.

The requirements were also checked for feasibility against the chosen technology stack. The React frontend, Express backend, Sequelize ORM, MySQL database, JWT authentication, and REST API structure support the required functionality. Automated backend tests using Jest and Supertest support requirement verification.

5.5 Requirement Specification and Documentation

The final requirements are documented in this SRS using consistent language and numbered sections. The specification avoids unnecessary implementation detail where possible, but it identifies important system behavior and data relationships. Requirements are written to be understandable by developers, testers, project guides, and evaluators.

The document also supports traceability because each major requirement maps to visible modules in the project, such as authentication, doctor availability, appointment management, medical records, prescriptions, notifications, and reports. This makes the SRS useful during design, implementation, testing, and viva preparation.

6 Functional Requirements

6.1 Patient Module

The system shall allow patients to register and log in securely using email and password credentials. After login, patients shall access a patient dashboard that displays upcoming appointments, recent medical records, and available doctors. The system shall store patient profile information such as age, gender, phone, address, and blood group.

The system shall allow patients to view doctors and filter them by name or specialization. Patients shall book appointments only from available doctor slots stored in the database. The system shall prevent bookings for past dates and shall prevent duplicate bookings for the same doctor, date, and time slot.

Patients shall be able to view, reschedule, cancel, and delete their own appointments. Rescheduling shall use available slots for the selected doctor. Cancelled appointments shall remain visible with cancelled status, while deleted appointments shall be removed from both the patient list and the doctor dashboard because the appointment record is deleted from the database.

Patients shall be able to view their medical records and prescriptions. The system shall display visit date, diagnosis, treatment, notes, doctor name, medicines, and dosage instructions. Patients shall also be able to print a prescription sheet from the record view.

6.2 Doctor Module

The system shall allow doctors to register and log in securely. Doctor registration shall store specialization, qualification, experience, and consultation fee. After login, doctors shall access a doctor dashboard that displays appointment statistics, upcoming visits, availability slots, and recent medical records.

Doctors shall be able to create availability slots by selecting a date, start time, and end time. Doctors shall also be able to delete open availability slots. The system shall prevent deletion of a slot that already has a booked appointment, so existing patient bookings remain protected.

Doctors shall be able to view upcoming booked appointments assigned to them. For each appointment, the system shall display doctor details, patient details, appointment date, appointment time, status, and reason. Doctors shall be able to complete a visit by entering diagnosis, treatment, notes, and visit date.

During visit completion, doctors may optionally create a prescription by entering medicines and dosage instructions. When the visit is completed, the system shall create a medical record, create the prescription if provided, and update the appointment status to completed in the database.

6.3 Administrator Module

The system shall allow administrators to log in securely and access an admin dashboard. Administrators shall be able to view summary information about patients, doctors, active appointments, medical records, and appointment status counts. The system shall also provide reports grouped by specialization so that hospital activity can be monitored.

Administrators shall have broader read access than patients and doctors. While patients are restricted to their own records and doctors are restricted to their assigned records, administrators may view system-level summaries for operational monitoring.

6.4 Authentication and Authorization

The system shall authenticate users using secure password verification. Passwords shall be stored as bcrypt hashes. After successful login or registration, the system shall issue a JWT token. Protected backend routes shall validate the token, identify the user, and apply role-based access checks.

Role-based authorization shall ensure that patients cannot create medical records, doctors cannot book patient appointments from the patient endpoint, and users cannot access records outside their permitted scope. These controls protect healthcare privacy and preserve correct workflow behavior.

6.5 Appointment Management

The system shall maintain appointments with patient, doctor, appointment date, appointment time, reason, and status. The allowed appointment statuses shall include booked, completed, and cancelled. The system shall support booking, listing, rescheduling, cancellation, deletion, and completion.

Appointment booking and rescheduling shall require a valid availability slot. The system shall check for conflicts before creating or updating a booked appointment. The doctor dashboard shall show only the doctor’s appointments, and the patient dashboard shall show only the patient’s appointments.

6.6 Medical Record and Prescription Management

The system shall allow doctors to create medical records for patients. Each medical record shall contain patient ID, doctor ID, diagnosis, treatment, notes, and visit date. Medical records shall be linked to the correct patient and doctor. Patients shall be able to view only their own medical records.

The system shall allow doctors to create prescriptions linked to medical records. Each prescription shall contain medical record ID, patient ID, doctor ID, medicines, and dosage instructions. The system shall verify that the prescription matches the patient and doctor of the selected medical record.

6.7 Notification and Reporting

The system shall create notifications for appointment and record-related events. Patients and doctors shall be able to view their own notifications and mark them as read. Notifications improve communication and reduce the chance of missed appointments or unnoticed updates.

The system shall provide summary reports for administrators. Reports shall include counts of booked, completed, and cancelled appointments, as well as doctor counts grouped by specialization. These reports support hospital monitoring and academic demonstration of analytics functionality.

7 Non-Functional Requirements

7.1 Performance

The system should provide fast response times for common actions such as login, doctor listing, appointment booking, appointment listing, and medical record viewing. Database indexes and query filters should support common searches by patient, doctor, date, and status.

7.2 Security

The system should protect user accounts and medical data using secure authentication, bcrypt password hashing, JWT validation, and role-based authorization. Sensitive routes should not be accessible without a valid token.

7.3 Privacy

Patients should access only their own appointments, records, prescriptions, and notifications. Doctors should access only appointments and records assigned to their doctor profile. Administrators should use broader visibility only for monitoring and reporting purposes.

7.4 Reliability and Availability

The system should handle failed requests gracefully and show meaningful error messages. Backend validation should prevent invalid state transitions, unavailable slot bookings, duplicate appointment conflicts, and unauthorized changes.

7.5 Usability

The user interface should be simple, role-specific, and easy to navigate. Patients should be able to book, reschedule, cancel, delete, and view records without technical knowledge. Doctors should be able to manage slots and complete visits from a single dashboard.

7.6 Scalability and Maintainability

The system should be modular so that features can be extended without major rewrites. The frontend should use reusable components and API clients. The backend should separate routes, controllers, middleware, models, and utility functions.

7.7 Testability

Backend behavior should be verified using automated tests. Authentication, appointment booking, deletion, availability, record creation, prescription creation, and role-based access should be covered by route-level tests wherever possible.

8 Constraints

The system depends on reliable internet connectivity and a running backend API. Since the system stores healthcare-related information, access control and privacy rules must be enforced carefully. The current version does not include payment gateway integration, video consultation, lab integration, insurance claim processing, or full hospital ERP inventory and billing.

The system also depends on the correctness of doctor availability data. Patients can book only the slots created by doctors. If a doctor has not added availability, patients will not see appointment times for that doctor.

9 Hardware and Software Requirements

The successful implementation and operation of the Smart Hospital Appointment and Record Management System require a suitable combination of hardware and software resources. These requirements ensure that the system can run reliably, support concurrent users, protect data, and provide a responsive user experience.

9.1 Hardware Requirements

The hardware infrastructure must support web application hosting, backend request processing, database storage, and browser-based access from client devices. The requirements can be divided into server-side, client-side, and networking hardware.

9.1.1 Server-Side Hardware Requirements

The backend server should have sufficient processing power and memory to handle authentication, appointment requests, database operations, notification logic, and report generation. For production use, the server may be hosted on a cloud platform or dedicated hospital infrastructure.

Key server-side hardware requirements include:

- Multi-core processor for concurrent request handling.
- Adequate RAM for Node.js services, database connections, and caching if added later.
- SSD storage for fast database access and application deployment.
- Backup storage for database recovery and data protection.
- Optional load balancer for future multi-server deployment.

9.1.2 Client-Side Hardware Requirements

Client-side hardware includes devices used by patients, doctors, and administrators. Since the project is a web application, users can access the system through modern browsers on laptops, desktops, tablets, or smartphones.

- Patient devices: smartphone, tablet, laptop, or desktop with internet connectivity.
- Doctor devices: laptop, desktop, or tablet for dashboard and record entry.
- Administrator devices: laptop or desktop for reports and monitoring.

9.1.3 Networking Hardware Requirements

Networking infrastructure is required for communication between the frontend, backend API, and database. Stable connectivity is important because appointment booking and medical record access depend on real-time server communication.

- Reliable internet connectivity for client devices.
- Secure local or cloud network for backend and database communication.
- Firewall or network access rules to protect backend and database services.

9.2 Software Requirements

Software requirements define the operating environment, development platform, database system, browser support, and third-party tools needed for implementation and operation.

9.2.1 Server-Side Software Requirements

The backend software environment handles business logic, authentication, authorization, persistence, and reporting. The project uses a RESTful API structure and a relational database.

- Operating System: macOS, Linux, or Windows for development; Linux preferred for deployment.
- Runtime: Node.js for backend execution.
- Backend Framework: Express.js.
- Database Management System: MySQL.
- ORM: Sequelize for model definitions and database interaction.
- Authentication Tools: JWT and bcrypt.
- Testing Tools: Jest and Supertest.

9.2.2 Client-Side Software Requirements

The frontend software environment provides the graphical user interface for all roles. It should support modern browsers and responsive layouts.

- Frontend Library: React.
- Build Tool: Vite.
- Styling: Tailwind CSS.
- Routing: React Router.

- HTTP Client: Axios.
- Browsers: Chrome, Firefox, Safari, Edge, or other modern browsers.

9.2.3 Third-Party Software and API Integrations

The current project uses internal REST APIs for the main hospital workflows. It is designed so that additional integrations can be added later if required.

- Email or SMTP service for appointment and record notifications.
- Cloud hosting platform for frontend and backend deployment.
- Database backup tools for MySQL data protection.
- Optional payment gateway APIs for future consultation fee payments.
- Optional video consultation APIs for future telemedicine support.

9.2.4 Security and Support Software

Security and support software are needed to protect sensitive healthcare data and maintain operational stability.

- HTTPS certificates for encrypted communication in deployment.
- Authentication and authorization middleware.
- Database backup and restore tools.
- Logging and monitoring tools for backend health and error tracking.
- Firewall or access-control configuration for database protection.

10 Conclusion

This paper presented a detailed Software Requirement Specification for a Smart Hospital Appointment and Record Management System. The specification captures the purpose, scope, stakeholders, requirement engineering approach, functional requirements, non-functional requirements, constraints, and hardware and software requirements of the proposed system.

The system provides practical healthcare workflows such as real doctor availability, patient appointment booking, rescheduling, cancellation, deletion, doctor visit completion, medical record creation, prescription management, notifications, and administrative reporting. The resulting SRS provides a strong foundation for design, implementation, testing, evaluation, and future enhancement of the project.